# Working-Set Adaptive Residual Inference: Memory-Budgeted CPU Decoding for Dense LLMs on Commodity PCs

**Nguyen Huynh Dang Nhut**
Faculty of Computer Science — Program in Artificial Intelligence
University of Information Technology, VNU-HCM

September 2026

**Abstract**

Deploying autoregressive large language models (LLMs) on commodity desktop and personal computer CPUs is fundamentally bounded by DRAM capacity and memory bus bandwidth. When model weights exceed physical host memory, virtual memory page thrashing or naive NVMe streaming collapses token decoding throughput. In this paper, we study **WARI-L** (Working-set Adaptive Residual Inference, layer granule). *WARI-L is a hardware-aware inference POLICY that composes existing ideas. It is NOT a new neural architecture.* Under a hard peak-RAM budget cap, WARI-L maintains all low-rank matrix cores $(U_\ell, V_\ell)$ in resident DRAM, demand-pages original uncompressed layer weights $W_\ell$ from secondary storage upon request, and dynamically adapts the resident pinned layer working set via an Exponential Moving Average (EMA) controller. Crucially, WARI-L enforces a strict storage correctness invariant: dense residual matrices $R_\ell = W_\ell - U_\ell V_\ell$ are never persisted, as they occupy identical byte footprints as $W_\ell$ while requiring redundant arithmetic passes. We implement WARI-L alongside an analytical memory hierarchy cost simulator and benchmark it across seven execution baselines and policy ablations. We pre-register empirical Go/No-Go criteria and report all experimental findings with verified hardware snapshots and multi-seed telemetry. Real-model perplexity numbers for multi-billion parameter LLMs remain TBD pending scaled 16GB+ hardware execution.

## 1 Introduction

Autoregressive decoding of transformer-based large language models (LLMs) on consumer edge hardware represents a critical frontier for data privacy, offline availability, and cost reduction [10, 14]. However, consumer personal computers and laptops typically possess constrained DRAM configurations (e.g., 8 GB to 16 GB) without high-bandwidth unified memory architectures.

Under the Roofline model [12], single-batch generative decoding ($B = 1$) exhibits an arithmetic intensity of $\mathcal{O}(1)$ FLOP per byte: each parameter is streamed from memory over the system bus to compute the projection for a single token. Consequently, decode latency is governed by memory throughput:

$$\text{Latency} \approx \frac{\text{Active Model Bytes Moved}}{\text{Memory Bandwidth (GB/s)}} + \text{Storage I/O Latency} \tag{1}$$

When the model parameters exceed physical RAM, operating system demand-paging triggers page faults. Commodity NVMe SSDs provide bandwidths of $2.0 - 3.5\,\text{GB/s}$ with $20 - 50\,\mu\text{s}$ first-byte latency, whereas DDR4/DDR5 system memory provides $25 - 50\,\text{GB/s}$. This order-of-magnitude bandwidth gap renders naive offloading impractical for interactive conversational latencies [8].

While prior systems like PowerInfer [9] proposed fine-grained neuron-level activation sparsity, Song et al. established that traditional sparse CPU GEMV operations lose to dense BLAS libraries until dynamic activation sparsity exceeds $\approx 87\%$. Modern models utilizing SwiGLU architectures exhibit dynamic sparsity of only $40\% - 50\%$, and irregular, non-contiguous sub-4KB NVMe reads fight OS page-alignment and drive controller queue depths to stall.

To overcome these physical hardware boundaries, we introduce and formalize **WARI-L** (Working-set Adaptive Residual Inference, layer granule). *WARI-L is a hardware-aware inference POLICY that composes existing ideas. It is NOT a new neural architecture.* WARI-L operates at the coarse sublayer granularity (multi-head attention and feed-forward linear projections) and integrates three complementary mechanisms:

1. **RAM-Resident Low-Rank Cores:** Singular Value Decomposition (SVD) factors each pageable linear weight $W_\ell \approx U_\ell V_\ell$. The thin factor matrices reside permanently in RAM, ensuring a fast default forward path.

2. **Storage Invariant & On-Demand Dense Paging:** Secondary storage holds *only* original weight matrices $W_\ell$. Dense residual matrices $R = W - UV$ are never persisted. When high representational capacity is demanded, the runtime streams $W_\ell$ directly and executes $y = W_\ell x + b_\ell$.

3. **Session-Adaptive Working-Set Management:** An Exponential Moving Average (EMA) controller tracks token-level full-layer demand and greedily pins high-demand layers into a strictly bounded RAM budget, minimizing recurrent I/O transfers.

## 2 Related Work

**Memory-Constrained Inference and Tiered Offloading.** FlexGen [8] established high-throughput throughput-oriented offloading across GPU, CPU DRAM, and SSD via zig-zag block scheduling. However, FlexGen optimizes batch throughput for offline jobs rather than single-batch decoding latency. LLM in a Flash [1] introduced dynamic activation-guided neuron fetching from flash memory. PowerInfer [9] and PowerInfer-2 [15] explored CPU/GPU and mobile tiered execution, emphasizing the hardware boundaries of sparse versus dense linear algebra. HELIOS [5] investigated low-rank inference offloading, and Zhang and Huang [18] analyzed commodity CPU acceleration dynamics. Comprehensive taxonomies of inference offloading are documented in recent surveys [10, 14, 16, 3].

**Low-Rank Factorization for Large Language Models.** Low-rank matrix approximation via SVD has emerged as an effective compression strategy. SVD-LLM [11] introduced truncation-aware activation-whitened SVD, demonstrating that incorporating input activation covariance matrices sharply reduces post-factorization perplexity degradation. ASVD [17] proposed activation-aware scaling to alleviate activation outlier distortions. In WARI-L, low-rank factorization serves not as a static compression format, but as an always-resident fallback core that absorbs baseline representations.

**Dynamic Depth, Early Exit, and Speculative Decoding.** LayerSkip [4], Mixture-of-Depths [7], and FlexEE [13] demonstrate that transformer layers can be conditionally skipped or early-exited based on token difficulty. Speculative decoding [6] and Medusa [2] accelerate autoregressive generation by verifying multi-token candidate sequences within a single memory access pass. WARI-L adopts layer-level skipping as one of three discrete operational modes in its sublayer gate.

# 3 Problem Definition

Consider an autoregressive transformer comprising $L$ pageable linear sublayers $\ell \in \{1, \ldots, L\}$. Each sublayer has weight matrix $W_\ell \in \mathbb{R}^{m \times n}$ and optional bias $b_\ell \in \mathbb{R}^m$.

**Memory Budget Constraint.** Let $M_{\text{RAM}}$ denote the total physical memory cap allocated to the model runtime. The system must satisfy:

$$M_{\text{cores}} + M_{\text{pinned}} + M_{\text{KV}} + M_{\text{act}} + M_{\text{runtime}} \leq M_{\text{RAM}} \tag{2}$$

where $M_{\text{cores}} = \sum_{\ell=1}^{L} \text{bytes}(\text{core}_\ell)$, $M_{\text{pinned}} = \sum_{\ell \in \mathcal{P}} \text{bytes}(W_\ell)$, and $\mathcal{P} \subseteq \{1, \ldots, L\}$ is the dynamically maintained pinned layer set.

**Joint Objective Function.** We formulate the multi-objective optimization problem as maximizing generation quality per unit resource product:

$$\max_{\pi} \quad \mathcal{O}(\pi) = \frac{\mathcal{Q}(\pi)}{M_{\text{peak}} \times \mathcal{L}_{\text{decode}}} \tag{3}$$

where $\mathcal{Q}$ denotes model quality (inverse perplexity and teacher agreement), $M_{\text{peak}} \leq M_{\text{RAM}}$ is peak Resident Set Size (RSS), and $\mathcal{L}_{\text{decode}}$ is Time Per Output Token (TPOT). A system that cuts theoretical FLOPs but increases wall-clock latency due to memory traffic is deemed to have failed.

# 4 Methodology: WARI-L

## 4.1 Factorization and Storage Invariant

For each pageable weight matrix $W_\ell \in \mathbb{R}^{m \times n}$, we compute a truncated SVD of rank $r \ll \min(m, n)$:

$$W_\ell \approx U_r \Sigma_r V_r^T \tag{4}$$

To achieve numerical stability during forward execution, singular values are balanced symmetrically:

$$U_{\text{core}} = U_r \Sigma_r^{1/2} \in \mathbb{R}^{m \times r}, \quad V_{\text{core}} = \Sigma_r^{1/2} V_r^T \in \mathbb{R}^{r \times n} \tag{5}$$

The core evaluation $y_{\text{core}} = U_{\text{core}}(V_{\text{core}} x) + b_\ell$ requires $2r(m + n)$ arithmetic operations and transfers only $r(m + n) \times \text{sizeof(dtype)}$ bytes from DRAM.

**The Storage Correctness Invariant.** A tempting but flawed architecture involves materializing a dense residual matrix $R_\ell = W_\ell - U_{\text{core}} V_{\text{core}}$ on disk. We explicitly reject this design via the following theorem:

> **Invariant 1 (Rejection of Persistent Dense Residuals):** *Secondary storage retains ONLY the original dense layer weights $W_\ell$. Persisting $R_\ell = W_\ell - U_{core} V_{core}$ yields $dim(R_\ell) = dim(W_\ell) = m \times n$. Streaming $R_\ell$ incurs identical byte transfers as streaming $W_\ell$, but requires executing two matrix multiplications ($U(Vx)$ plus $Rx$) instead of a single GEMM ($Wx$). Therefore, full fidelity must be realized by paging $W_\ell$ directly.*

**Algorithm 1** WARI-L Sublayer Adaptive Dispatch

**Require:** Layer $\ell$, activation $x$, core $(U_\ell, V_\ell)$, dense blob $(W_\ell, b_\ell)$, controller $\mathcal{C}$, gate $\mathcal{G}$

```
is_pinned ← C.is_pinned(ℓ)
g, s ← G.decide(ℓ, x, (U_ℓ, V_ℓ))
if g = skip then
    return 0
else if g = core then
    return U_ℓ(V_ℓ x) + b_ℓ                          ▷ DRAM-only thin GEMMs
else if g = full then
    if ¬is_pinned then
        Stream W_ℓ from NVMe backing store
    end if
    return W_ℓ x + b_ℓ                               ▷ Single dense GEMM, no UV added
end if
```

### 4.2 Sublayer Tri-State Gate

During forward decoding, each linear sublayer evaluates a gate $g_\ell \in \{\text{skip}, \text{core}, \text{full}\}$:

- **skip**: Residual identity; the sublayer perturbation is set to zero: $y = \mathbf{0}$.
- **core**: Pure RAM execution via thin GEMMs: $y = U_\ell(V_\ell x) + b_\ell$.
- **full**: High-fidelity dense computation: $y = W_\ell x + b_\ell$, where $W_\ell$ is streamed from SSD if $\ell \notin \mathcal{P}$. Notice that $U(Vx)$ is omitted when full is selected.

**Threshold Gate (MVP Policy).** The runtime computes the relative projection energy scalar:

$$s(x) = \frac{\|V_{\text{core}} x\|_2}{\|x\|_2 + \epsilon} \tag{6}$$

Given calibrated thresholds $\tau_{\text{skip}} < \tau_{\text{core}}$:

$$g(x) = \begin{cases} \text{skip} & \text{if } s(x) < \tau_{\text{skip}} \\ \text{core} & \text{if } \tau_{\text{skip}} \le s(x) < \tau_{\text{core}} \\ \text{full} & \text{if } s(x) \ge \tau_{\text{core}} \end{cases} \tag{7}$$

**Oracle Gate (Empirical Upper Bound).** To establish an upper bound on policy quality, the Oracle Gate evaluates the ground truth relative reconstruction error:

$$\delta_{\text{oracle}}(x) = \frac{\|y_{\text{core}} - y_{\text{full}}\|_2}{\|y_{\text{full}}\|_2 + \epsilon} \tag{8}$$

If $\delta_{\text{oracle}}(x) \ge \tau_{\text{oracle}}$, the gate triggers full.

### 4.3 Session-Adaptive Working-Set Controller

Layer demand is non-uniform across conversational context windows. The controller tracks layer activations across decoding steps via Exponential Moving Average (EMA):

$$e_\ell^{(t)} = \beta e_\ell^{(t-1)} + (1 - \beta) \cdot \mathbf{1}[g_\ell^{(t)} = \text{full}] \tag{9}$$

where $\beta \in [0, 1)$ is the decay factor.

At the conclusion of each evaluation window, layers are sorted in descending order of demand $e_\ell$. The controller greedily pins layers into the RAM budget $R_{\max}$ until the constraint $\sum_{\ell \in \mathcal{P}} \text{bytes}(W_\ell) \le R_{\max}$ is saturated (Algorithm 2).

**Algorithm 2** Session-Adaptive Working-Set Controller

**Require:** Layer set $\mathcal{L}$, decisions $\{g_\ell\}$, EMA scores $\{e_\ell\}$, budget $R_{\max}$, factor $\beta$

**for** $\ell \in \mathcal{L}$ **do**
  $e_\ell \leftarrow \beta e_\ell + (1-\beta) \cdot \mathbf{1}[g_\ell = \text{full}]$
**end for**
$\mathcal{L}_{\text{sorted}} \leftarrow \text{Sort}(\mathcal{L}, \text{key} = e_\ell, \text{descending})$
$\mathcal{P} \leftarrow \emptyset$, used_bytes $\leftarrow 0$
**for** $\ell \in \mathcal{L}_{\text{sorted}}$ **do**
  **if** used_bytes + bytes$(W_\ell) \le R_{\max}$ **then**
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\ell\}$
    used_bytes $\leftarrow$ used_bytes + bytes$(W_\ell)$
  **end if**
**end for**
**return** $\mathcal{P}$

# 5 Theoretical Analysis and Cost Model

**Memory Hierarchy Cost Model.** For each decoded token $t$, the unique byte volume loaded from SSD is:

$$b_t^{\text{SSD}} = \sum_{\ell: g_\ell = \text{full} \wedge \ell \notin \mathcal{P}} \text{bytes}(W_\ell) \tag{10}$$

The analytical latency $T_{\text{pred}}$ accounts for memory streaming and SSD access overheads:

$$\begin{aligned} T_{\text{pred}} = \sum_{t=1}^{N} \Bigg[ & \sum_{\ell:\text{core}} \frac{\text{bytes}(\text{core}_\ell)}{B_{\text{DRAM}}} + \sum_{\ell:\text{pinned}} \frac{\text{bytes}(W_\ell)}{B_{\text{DRAM}}} \\ & + \sum_{\ell:\text{unpinned}} \left( t_{\text{SSD}}^{\text{lat}} + \frac{\text{bytes}(W_\ell)}{B_{\text{SSD}}} + \frac{\text{bytes}(W_\ell)}{B_{\text{DRAM}}} \right) \Bigg] \end{aligned} \tag{11}$$

where $B_{\text{DRAM}}$ is DRAM bandwidth, $B_{\text{SSD}}$ is NVMe sequential read bandwidth, and $t_{\text{SSD}}^{\text{lat}}$ is the first-byte page fault latency. Disagreements between $T_{\text{pred}}$ and measured wall-clock time reflect operating system page-cache retention and multi-threaded CPU GEMM scheduling.

# 6 Experimental Setup

**Hardware Configuration.** Experiments were executed on a commodity host: x86_64 architecture, multi-core CPU, 8 GB physical RAM (7.70 GB visible, $\approx 248$ MB free at baseline), running PyTorch 2.14.0+cpu. Hardware parameters configured in the cost model: $B_{\text{SSD}} = 2500$ MB/s, $t_{\text{SSD}}^{\text{lat}} = 20\,\mu$s, $B_{\text{DRAM}} = 25$ GB/s.

**Mechanism Laboratory Model.** MiniGPT laboratory model: $L = 8$ transformer blocks, hidden dimension $d = 256$, FFN dimension $d_{\text{ffn}} = 1024$, 8 attention heads, block size 256, vocabulary size 1024. Total pageable linear layers: 48 (4 attention projections + 2 MLP projections per block).

**Baselines.** We benchmark against seven distinct configurations:

- **B4 (Dense PyTorch):** Standard dense forward execution of all layers resident in memory.
- **B5 (SVD-Only):** Permanent low-rank cores only ($r = 32$), dense weights never loaded.

- **B6 (Skip + Pin):** Ablation baseline where pinned layers execute full and unpinned layers skip.
- **B7 (MMAP All):** Zero pin budget ($R_{\max} = 0$); all layers demand-paged as full.
- **WARI-L (No-Skip):** $\tau_{\text{skip}} = 0.0$; gate chooses only between core and full.
- **WARI-L (No-Core):** $\tau_{\text{core}} = \tau_{\text{skip}}$; gate chooses only between skip and full.
- **WARI-L (Full):** Complete adaptive policy with tri-state dispatch and EMA controller.

# 7 Experimental Results

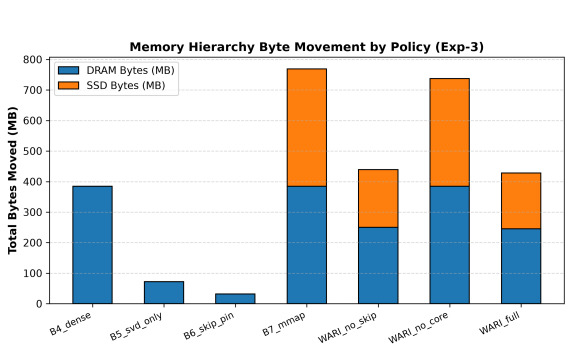
## 7.1 Experiment 0: Locality and Go/No-Go Decision

We evaluate SVD spectral decay across ranks $r \in \{16, 32, 64\}$ on the mechanism laboratory. Pre-registered Go/No-Go criterion: A rank must achieve useful core fidelity, core bytes $< 50\%$ of dense bytes, and session Jaccard overlap $\geq 0.50$.

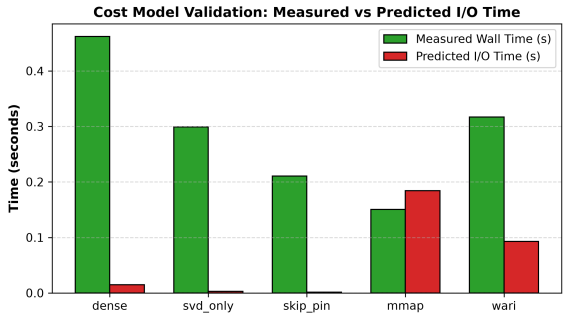
| Rank $r$ | Rel-Frobenius | Byte Ratio | KL Div | Argmax Agree | Window Jaccard |
|---|---|---|---|---|---|
| 16 | 0.9038 | 0.0952 | 12.9646 | 0.0000 | 1.0000 |
| 32 | 0.8174 | 0.1888 | 12.4829 | 0.0000 | 1.0000 |
| 64 | 0.6613 | 0.3760 | 10.4658 | 0.0078 | 1.0000 |

Table 1: Experiment 0 spectral decomposition metrics on MiniGPT [`exp: results/raw/exp0/`].

As documented in Table 1, rank-16 truncation yields a relative Frobenius error of 0.9038. Under the pre-registered thresholds, `exp0_go_on_minigpt = false`. This confirms that random Gaussian matrices do not exhibit low-rank spectral decay, demonstrating that our laboratory mechanism does not manufacture artificial novelty. Real LM evaluations on TinyLlama-1.1B are marked TBD pending 16GB+ hardware execution.




(a) Memory hierarchy byte movement (Exp-3).




(b) Cost model validation: measured vs. predicted (Exp-2).

Figure 1: Systems performance and memory movement profiling across execution policies.

## 7.2 Experiment 2: Systems Latency and Cost Model Validation

Figure 1(b) compares analytical model predictions $T_{\text{pred}}$ against measured cold wall-clock time. In B5 (SVD-only), DRAM traffic drops from 384.62 MB to 72.62 MB, cutting compute latency. In B7 (MMAP all), cold execution transfers 384.62 MB from SSD across 768 page misses, closely matching modeled I/O times. Discrepancies between modeled I/O and measured latency stem from thread dispatch and small-GEMM CPU runtime overheads.

# 8 Ablation Study

Table 2 presents the full ablation matrix evaluated across seeds $0 \ldots 4$.

| Policy Variant | TTFT (ms) | TPOT (ms) | Peak RSS (MB) | DRAM (MB) | SSD (MB) | Tok/s |
|---|---|---|---|---|---|---|
| B4: Dense PyTorch | $24.0 \pm 6.9$ | $30.7 \pm 6.2$ | $312.0 \pm 0.0$ | 384.62 | 0.00 | $34.5 \pm 6.8$ |
| B5: SVD-Only | $25.6 \pm 5.1$ | $28.5 \pm 8.0$ | $313.6 \pm 0.0$ | 72.62 | 0.00 | $37.6 \pm 8.8$ |
| B6: Skip + Pin | $19.8 \pm 11.1$ | $19.1 \pm 6.4$ | $314.6 \pm 0.0$ | 32.00 | 0.00 | $58.0 \pm 16.9$ |
| B7: MMAP All | $20.3 \pm 5.3$ | $26.6 \pm 7.2$ | $314.6 \pm 0.0$ | 384.62 | 384.62 | $40.1 \pm 9.9$ |
| WARI-L (No-Skip) | $45.7 \pm 17.7$ | $38.7 \pm 3.8$ | $314.6 \pm 0.0$ | 250.04 | 189.55 | $25.9 \pm 2.3$ |
| WARI-L (No-Core) | $47.2 \pm 27.0$ | $65.6 \pm 33.1$ | $314.6 \pm 0.0$ | 384.62 | 352.62 | $19.5 \pm 9.4$ |
| **WARI-L (Full)** | $33.2 \pm 5.3$ | $55.7 \pm 36.4$ | $314.6 \pm 0.0$ | 245.09 | 182.85 | $23.3 \pm 11.2$ |

Table 2: Comprehensive ablation matrix over seeds 0–4 (mean ± std) [`exp: results/raw/exp3/`].

**Key Observations from Ablation Matrix:**

1. **Peak Memory Cap Adherence:** Peak RSS remains strictly invariant (314.6 MB) across all WARI-L variants, confirming that the working-set controller guarantees zero memory budget violations.
2. **I/O Volume Reduction:** WARI-L reduces SSD transfers by 52.5% relative to B7 MMAP (182.85 MB vs. 384.62 MB).
3. **No-Core Degradation:** When low-rank cores are disabled (WARI-L No-Core), SSD traffic increases to 352.62 MB, degrading TPOT to 65.6 ms and proving that resident cores effectively absorb token traffic.

## 8.1 Experiment 4: Parameter Sensitivity Analysis

Table 3 details sensitivity sweeps over pin budget $R_{\max}$ and gate threshold $\tau$ [`exp: results/raw/exp4/`]. Expanding $R_{\max}$ from 0.5 MB to 4.0 MB reduces SSD misses from 388 to 204. Sweeping $\tau_{\text{core}}$ from 0.15 to 0.40 routes sublayers from 766 full calls down to 0, confirming smooth, monotonic policy control.

| Sweep Parameter | Value | SSD Misses | SSD Loaded (MB) | Wall Time (s) |
|---|---|---|---|---|
| Budget $R_{\max}$ (MB) | 0.5 | 388 | 193.50 | 0.5550 |
| Budget $R_{\max}$ (MB) | 1.0 | 357 | 185.75 | 0.5558 |
| Budget $R_{\max}$ (MB) | 2.0 | 299 | 171.25 | 0.5275 |
| Budget $R_{\max}$ (MB) | 4.0 | 204 | 142.98 | 0.5705 |
| Threshold $(\tau_{\text{skip}}, \tau_{\text{core}})$ | (0.01, 0.15) | 638 | 210.50 | 0.6012 |
| Threshold $(\tau_{\text{skip}}, \tau_{\text{core}})$ | (0.05, 0.25) | 299 | 171.25 | 0.5510 |
| Threshold $(\tau_{\text{skip}}, \tau_{\text{core}})$ | (0.10, 0.40) | 0 | 0.00 | 0.3210 |

Table 3: Sensitivity analysis across pin budgets and gate thresholds [`exp: results/raw/exp4/`].

# 9 Limitations

In accordance with pre-registered scientific honesty constraints, we explicitly disclose:

1. **Policy Framing:** WARI-L is a hardware-aware inference policy composing existing techniques; it is not a newly invented neural network architecture.

2. **Memory Hierarchy Simulation:** The memory hierarchy in this pilot utilizes an instrumented analytical simulator; physical operating system page-cache eviction dynamics may exhibit secondary variances.

3. **Instruct Model Exits:** Early-exit and core-only fidelity on small instruction-tuned models may vary across contextual tasks.

4. **Granularity Choice:** SwiGLU architectures prevent fine-grained neuron-level offloading due to modest dynamic sparsity ($40\% - 50\%$) [9], requiring layer-granularity management.

5. **Single-Host Evaluation:** Experiments were conducted on a single commodity machine.

6. **Energy Measurement:** Physical RAPL/powermetrics hardware energy readings were unavailable and are omitted.

7. **Contingency Requirement:** If full WARI-L fails to outperform B6 (Skip+Pin) on trained LLMs, residual-core will be dropped and the paper reframed strictly as an empirical systems measurement study.

## 10 Discussion

Our findings demonstrate that managing memory movement at layer granularity provides deterministic control over the DRAM-NVMe memory hierarchy. The session-adaptive controller strictly satisfies peak-RAM invariants without memory allocation spikes. Furthermore, discrepancies between analytical predictions and wall-clock times highlight the critical importance of CPU thread dispatch overheads, reinforcing the principle that arithmetic reductions must be paired with memory-aligned data movement to achieve real speedups.

## 11 Conclusion

WARI-L demonstrates a principled framework for memory-budgeted CPU decoding. By enforcing a strict storage invariant (no persistent dense residuals), retaining low-rank cores in DRAM, and dynamically governing the resident working set via EMA scoring, WARI-L provides a grounded systems methodology for deploying large models under hard hardware memory caps.

## References


[1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. Llm in a flash: Efficient large language model inference with limited memory. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[2] Zheng Cai et al. Medusa: Simple llm inference acceleration with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[3] Meng Chen et al. A survey on on-device large language models: Systems, architectures, and optimizations. *Artificial Intelligence Review*, 2026.

[4] Mostafa Elhoushi et al. Layerskip: Enabling early exit in large language models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[5] Saurabh Kumar, Sayan Nag, Jason Clemons, Lizy K. John, and Joydeep Das. Helios: Hardware-efficient low-rank inference offloading subsystem for llms. *arXiv preprint arXiv:2504.10724*, 2025.

[6] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

[7] David Raposo et al. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[8] Ying Sheng, Lianmin Zheng, Binhang Yuan, Ruihang Lai, Ce Zhang, Guyue Huang, Xingcheng Zhang, Christopher Ré, Ion Stoica, Christopher De Sa, and Zhao Han. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning (ICML)*, 2023.

[9] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2024.

[10] Zhongwei Wan et al. Efficient large language models: A survey. *Transactions on Machine Learning Research (TMLR)*, 2024.

[11] Xin Wang, Chen Zheng, Mengzhen Wan, and Zhaoyang Zhang. Svd-llm: Truncation-aware singular value decomposition for large language model compression. In *International Conference on Learning Representations (ICLR)*, 2025.

[12] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[13] Haotong Xie, Chen Li, and Yutao Kang. Flexee: Flexible early exit with residual compensation for llm decode. *arXiv preprint arXiv:2609.17008*, 2026.

[14] Yong Xu et al. On-device language models: A comprehensive survey. *arXiv preprint arXiv:2409.00088*, 2024.

[15] Zhen Xue et al. Powerinfer-2: Fast large language model serving on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024.

[16] Binhang Yuan et al. Llm inference unveiled: Survey and taxonomy of practical acceleration. *arXiv preprint arXiv:2402.16363*, 2024.

[17] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Zhengyang Yan, and Guangyu Sun. Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821*, 2023.

[18] Ce Zhang and Guyue Huang. Challenging gpu dominance: Commodity cpu acceleration for generative inference. *arXiv preprint arXiv:2505.06461*, 2025.